

Full-Time-Pad Symmetric Stream Cipher

Improved One-Time-Pad Encryption Scheme

Taha Canturk
kibnakanoto@protonmail.com

2024-05-20

Version 1.0

License to copy this document is granted provided it is identified as "Full-Time-Pad", in all material mentioning or referencing it.

Abstract

Essential properties in ciphers include computational efficiency, security, and economical feasibility. Encryption scheme based on one-time-pad (OTP) is secure, but there are two security risks: (1) a key cannot be reused and (2) plaintext length equals the key length which is costly & inefficient for long plaintexts. As stated by Claude Shannon, father of information theory [1], a secure cryptographic algorithm requires both confusion and diffusion. However OTP lacks the confusion and diffusion per each ciphertext. In this study, we proposed the **full-time-pad (FTP)**, a symmetric stream cipher that extends OTP by incorporating both diffusion and confusion. To achieve diffusion, symmetric key is permuted in its byte form using a constant permutation matrix. The permutation ensures that every 32-bit number is derived from different bytes of the key. Using modern hardware optimizations (Vectorization, Cache and Parallelization, this operation is computationally efficient. To achieve confusion, the key is manipulated in 32-bit segments using Modular **A**ddition, Bitwise **R**otations, and **X**or (**ARX**).

Contents

1	Introduction	1
1.1	Pre-requisite Terminology	2
1.2	Key Generation	2
1.3	Prerequisite Mathematics	3
1.3.1	Galois Field Addition	3
1.3.2	Vector Permutation	3
2	Security	4
2.1	NIST SP 800-22 - Statistical Test Suite - Official NIST Randomness Test . .	5
2.2	Brute-Force	5
2.2.1	Birthday Problem	5
2.2.2	Collision Attack	10
2.3	Joux's Attack: Multicollisions in Iterated Hash Functions	13
2.4	Related Key Attack	13
2.5	Time Based Attacks	13
2.6	Linear Cryptanalysis	14
2.7	Differential Cryptanalysis	15
2.8	Reverse Engineering the Transformation	17
2.9	Side-Channel Attacks	17
2.10	Collision-Resistance	17
2.10.1	Different Permutation Matrices	17
2.10.2	Number of Rounds	17
2.10.3	Constant - F_p - Prime Galois Field Size	17
2.10.4	Constant - r - Dynamic Rotation Constant	18
3	Hashing	19
3.1	Dynamic Matrix Permutation	20
3.1.1	Deravation	21
3.1.2	Dynamic Permutation Matrix Values	22
3.2	Confusion - ARX	22
3.2.1	A - Modular Addition	22
3.2.2	R - Bitwise Rotation	23
3.2.3	X - XOR	23
4	Performance	23
5	Cipher	24
5.1	Transformation - Version 1.0 - Prioritize Complexity	24
5.2	Transformation - Version 1.1 - Prioritize Speed & Complexity	25
5.3	Transformation - Version 2.0 - Prioritize Speed	25
5.4	dynamic_permutation	27
5.5	endian_8_to_32_arr	27
5.6	Key Reuse	27

5.6.1	Multi-Threading	27
5.7	Long Plaintexts	28
5.8	Encryption/Decryption operation	29

1 Introduction

Fast & efficient encryption algorithms are a desired concept for both large and small amounts of data. Single Instruction Multiple Data (SIMD) instructions in modern devices are a golden opportunity for hardware optimized cryptographic algorithms. Even though newer algorithms like Blake-3 use them, they don't maximize their effects. According to my benchmarking tests on FTP, most cache misses happen on the first permutation, next permutations are consistently faster which is why using multiple permutations are a worthwhile approach. This high-speed algorithm (3x faster than ChaCha20) makes use of modern hardware accelerations. SIMD Parallelization has been around since 1962 [2], they have been popularized since early 1990s [4]. Ever since then, SIMD exists in nearly all modern devices and we present that encryption algorithms should adapt to this modernization. One of the main reasons why multiple rounds of permutations aren't considered in previous works is because diffusion relied on transposing bytes located in RAM while FTP relies on cache and simd optimizations. In the Efficient Permutation Instructions for Fast Software Cryptography [3] publication from Princeton University, bit-level permutation optimization are introduced. Such algorithms would significantly reduce the number of rounds needed for a cryptographic algorithm such as ChaCha20 which includes 20 rounds of transformations yet FTP only requires 10 rounds. This is because of the byte-level permutation mixed with simple ARX operations. However hardware bit-level permutation optimizations aren't offered by personal devices yet so they cannot be used.

1.1 Pre-requisite Terminology

Key	32-byte random array that's transformed, then hashed before XORed with the plaintext to encrypt
Symmetric	Same key is used for encryption and decryption
Stream	Plaintext is encrypted without separating it into blocks
Plaintext	Plain data before encryption
Ciphertext	Encrypted plaintext
Cipher	Encryption algorithm. Plaintext is transformed into a ciphertext that can only be reversed with a key
Diffusion	plaintext/key is spread out in the ciphertext
Confusion	The ciphertext has no possible statistical analysis, or cryptanalysis to determine the plaintext
Bit	0 or 1. Smallest discrete unit for computation
Byte	8-bit number
Galois Field	Finite field where there are only limited number of numbers. Only prime galois fields (F_p) are used where size of the field is denoted by prime number p
Avalanche Effect	An aspect of diffusion. If smallest unit (1 bit) of data is changed, the ciphertext changes in an unrecognizable way.
Permutation	transposing (moving) elements of a vector without actually changing their values. A reversible operation

1.2 Key Generation

The 32-byte key should be generated using a cryptographically secure method, including but not limited to cryptographic random number generators and Elliptic Cryptography Diffie Hellman (ECDH) protocol with Hash-based Key Derivation Function (HKDF).

1.3 Prerequisite Mathematics

1.3.1 Galois Field Addition

In a prime galois field, where two variables (x, y) are added, $x, y \in \mathbb{F}_p$ where $\mathbb{F}_p$ denotes the finite field with a prime field size. There are finite number of elements in F_p which means that $x, y < p$.

The expression:

$$x + y = z$$

$$x, y, z \in \mathbb{F}_7$$

Implicitly denotes

$$x + y = z \pmod{7}$$

So all addition operations require modulo p.

In the context of this encryption algorithm, it's used in the tranformation algorithm as one of three operations (**ARX**).

1.3.2 Vector Permutation

Permutating a vector refers to transposing the elements in a vector. It is a reversible operation. For example, if for the vector: V

$$V = \begin{pmatrix} 0 & 1 & 2 & 3 \end{pmatrix}$$

and the permutation vector P :

$$P = \begin{pmatrix} 3 & 2 & 1 & 0 \end{pmatrix}$$

then the resulting matrix would be

$$V' = \begin{pmatrix} 3 & 2 & 1 & 0 \end{pmatrix}$$

If we look at V_0 (**0**), according to P_0 , it should be replaced with the 3rd value in P . so then $V'_0 = 3$. This doesn't mean that $V'_3 = 0$. This algorithm is demonstrated in dynamic_permutation section (5.4). . The permutation in this algorithm is used for the bytearray **key** and the 32-bit segmented **k**. In order to confuse (**ARX**) and diffuse (permute), **k** and **key** needs to point to the same memory address. Most efficient way to do this, would be using **reinterpret_cast** in C++ or using **unions** in C. Since these are low level approaches, endiannes of the system will affect the byte order. In a little-endian system, the data requires the following permutation before the transformation iterations:

$$V' = \begin{pmatrix} 3 & 2 & 1 & 0 & 7 & 6 & 5 & 4 & 11 & 10 & 9 & 8 & 15 & 14 & 13 & 12 & 19 & 18 & 17 & 16 & 23 & 22 & 21 & 20 & 27 & 26 & 25 & 24 & 31 & 30 & 29 & 28 \end{pmatrix}$$

This means that for little endian systems, the permutation matrix also needs to change. The little-endian permutation matrix derivation is given in the `test/perm.py`.

2 Security

In One-Time-Pad, key isn't reusable. Here is the proof:

```
let m1, m2 be 2 plaintexts
let k be the key
let c1 = m1 ⊕ k
let c2 = m2 ⊕ k
c1 ⊕ c2 = (m1 ⊕ k) ⊕ (m2 ⊕ k)
c1 ⊕ c2 = m1 ⊕ m2
```

Since the key is reused, the 2 ciphertexts XORed factor out the key since $k \oplus k = 0$. Using cryptanalysis, the 2 plaintexts can be found.

For $c_1 \oplus c_2 = m_1 \oplus m_2$ to not hold true, for each encryption, the key needs to be different. If k is transformed each time so that it has an avalanche effect. Even with no confusion, it would still be secure since $k' \oplus k \neq 0$ where k' is transformed key.

But there is another concern,

What if the plaintext and ciphertext are known (known plaintext attack), then it is possible to find k so don't use k without transformation, since $\text{plaintext} \oplus \text{ciphertext} = \text{key}$. So for each plaintext, key needs to be transformed irreversibly and it also requires confusion since if k' is found, k is still unknown but if k is found, then all instances of k'_n are known, which means that:

```
k'1 = hash(k + 1) where hash() is an irreversible transformation
k'2 = hash(k + 2)
c1 ⊕ c2 = (m1 ⊕ k'1) ⊕ (m2 ⊕ k'2)
c1 ⊕ c2 ≠ m1 ⊕ m2
m1 ⊕ c1 = k'1
m2 ⊕ c2 = k'2
k'1, k'2 are calculated using an irreversible hashing algorithm
```

∴ the Full-Time-Pad Cipher requires both diffusion and confusion

2.1 NIST SP 800-22 - Statistical Test Suite - Official NIST Randomness Test

using the python tool from https://github.com/stevenang/randomness_testsuite, all 3 versions of the transformation algorithm have been testing using a fixed key (0-31) and plaintext (0-31). Only one key was used. Per each 256-bits until the one millionth bit, the encryption index was incremented once per encryption. All 3 versions of the transformation function resulted in full-randomness for all 16-tests. This means that the slightest change in input (key/encryption index) provides a very good avalanche effect, and therefore has a proper avalanche effect.

2.2 Brute-Force

Due to the use of a galois field. The total number of combinations per 256-bit key isn't $a = 2^{256}$, but rather $b = 4294967291^8$ where $p = 4294967291$ for arithmetic in F_p and there are 8 32-bit numbers in a 256-bit key.

```
a = 11579208923731619542357098500868790785326998466564056403945758400791312963993610
b = 11579208815891833313151659776217239262857046546585679399233288413030729265712110
let Δ = a - b
Δ = 107839786229205438724651551522469951919978377004712469987760583698281510
```

So the difference Δ is a somewhat large integer. The number of combinations with a galois field is lower than without a galois field ($b < a$). This isn't a big concern as their difference measured exponentially is only around $2^{\log_2 \Delta} \approx 2^{229}$ which means that their difference is around 2^{229} , this is a negligible difference as the difference between 2^{230} and 2^{229} is also huge. $\therefore$ Using a galois field doesn't negatively impact number of combinations in terms of brute force as the total number of combinations when using a galois field vs not is a negligible amount

2.2.1 Birthday Problem

The birthday problem is a veridical paradox. It goes as follows: how many people are required so that there is more than 50% chance that at least 2 people have the same birthday. The answer is an unexpected 23 people.

In the context of this encryption algorithm, this means that brute-forcing using random inputs is more likely than your intuition predicts especially since a galois field is used (smaller number of total combinations):

```
let  $V_c$  be the number of combinations per key with order and no repetitions
let  $k$  be the number of keys needed for hash(key) to have a 50% chance to equal
another hash(key)
let  $V_t$  be the number of combinations per key with order and with repetitions
```

$$V_c = \frac{b!}{(b-k)!} = \frac{4294967291^8!}{(4294967291^8 - k)!}$$

$$V_t = b^k = 4294967291^{8^k}$$

$$f_A(b,k) = \frac{V_c}{V_t}$$

$$f_A(b,k) = \frac{\frac{b!}{(b-k)!}}{b^k}$$

$$f_B(b,k) = 1 - f_A(b,k) = 50\%$$

$$f_A(b,k) = 1 - 50\%$$

$$1 - 50\% = \frac{\frac{b!}{(b-k)!}}{b^k}$$

$$\frac{1}{2}b^k = \frac{b!}{(b-k)!}$$

$$\text{since } 50\% = \frac{1}{2}$$

$$\log_b \frac{1}{2}b^k = \log_b \frac{b!}{(b-k)!}$$

$$\log_b \frac{1}{2} + \log_b b^k = \log_b b! - \log_b (b-k)!$$

$$0 = \log_b b! - \log_b (b-k)! - \log_b \frac{1}{2} - k$$

$$\text{since } \log_b b^k = k$$

According to Ramanujan's Approximation:

$$\log_b b! \approx \frac{b \ln b - b + \frac{\ln \left[\frac{1}{\pi^3} + b(1+4b(1+2b)) \right]}{6} + \frac{\ln \pi}{2}}{\ln b}$$

And

$$\log_b(b-k)! \approx \frac{(b-k) \ln(b-k) - (b-k) + \frac{\ln \left[\frac{1}{\pi^3} + (b-k)(1+4(b-k)(1+2(b-k))) \right]}{6} + \frac{\ln \pi}{2}}{\ln b}$$

Recall:

$$0 = \log_b b! - \log_b(b-k)! - \log_b \frac{1}{2} - k \quad \text{isolate } \log_b(b-k)!$$

$$\log_b(b-k)! = \log_b b! - \log_b \frac{1}{2} - k$$

Combine both equations for $\log_b(b-k)!$:

$$\log_b b! - \log_b \frac{1}{2} - k \approx \frac{(b-k) \ln(b-k) - (b-k) + \frac{\ln \left[\frac{1}{\pi^3} + (b-k)(1+4(b-k)(1+2(b-k))) \right]}{6} + \frac{\ln \pi}{2}}{\ln b}$$

$$\begin{aligned} \frac{b \ln b - b + \frac{\ln \left[\frac{1}{\pi^3} + b(1+4b(1+2b)) \right]}{6} + \frac{\ln \pi}{2}}{\ln b} - \log_b \frac{1}{2} - k &\approx \frac{(b-k) \ln(b-k) - (b-k)}{\ln b} + \\ &+ \frac{\frac{\ln \left[\frac{1}{\pi^3} + (b-k)(1+4(b-k)(1+2(b-k))) \right]}{6} + \frac{\ln \pi}{2}}{\ln b} \end{aligned}$$

$$\frac{b \log b \cancel{b} + \frac{\ln \left[\frac{1}{\pi^3} + b(1+4b(1+2b)) \right]}{6} + \frac{\ln \pi}{2} - \ln b \log_b \frac{1}{2} - \ln bk}{\ln b} \approx \frac{(b-k) \ln(b-k) \cancel{b} + k}{\ln b} + \frac{\ln \left[\frac{1}{\pi^3} + (b-k)(1+4(b-k)(1+2(b-k))) \right]}{6} + \frac{\ln \pi}{2}$$

$$\text{let } C(b) = b \ln b + \frac{\ln \left[\frac{1}{\pi^3} + b(1+4b(1+2b)) \right]}{6} - \ln \frac{1}{2} \approx (b-k) \ln(b-k) + k + \ln bk + \frac{\ln \left[\frac{1}{\pi^3} + (b-k)(1+4(b-k)(1+2(b-k))) \right]}{6}$$

$$\text{let } g(b, k) = (b-k) \ln(b-k) + k + \ln bk + \frac{\ln \left[\frac{1}{\pi^3} + (b-k)(1+4(b-k)(1+2(b-k))) \right]}{6} - C(b) = 0$$

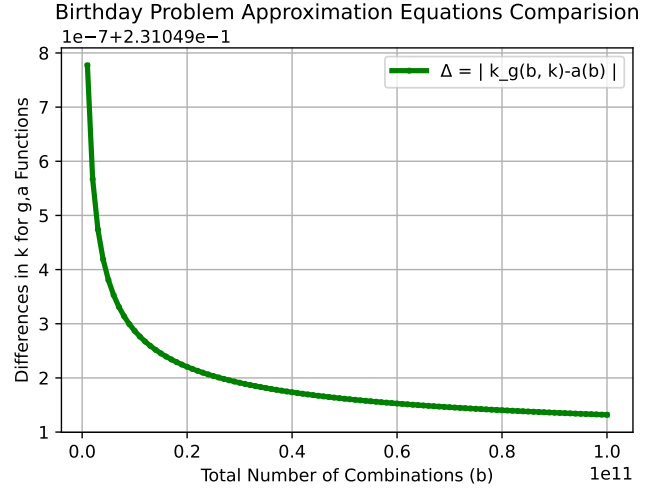
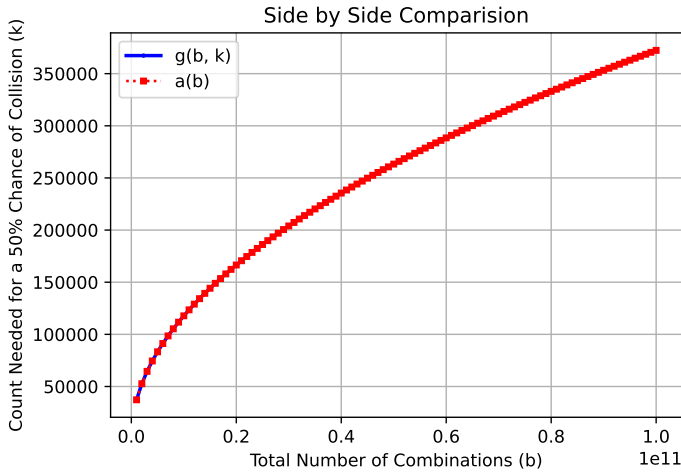
To verify the accuracy of this equation, we compared it to a proven accurate approximation equation [5].

$$a(b) = \frac{1}{2} + \sqrt{\frac{1}{4} + 2 \ln 2b}$$

And an accurate independent verification solution $f_\Gamma(b, k)$ to solve the f_B function defined above with Γ functions since $b, k \notin \mathbb{Z}$ for exact solution.

To compare functions a and g :

1. Provide both functions with inputs for parameter b that increments by an arbitrary integer (n) for a 100 iterations.
 2. Graph their values and the differences in k .
 3. Find k using another approved method. E.g. solving the birthday problem with f_Γ .
- (1.-2.): Increment b by $n=1,000,000,000$ for 100 iterations and graph (b, k) for g, a :



Graph Δ shows that the difference in k for g, a get smaller as $b \rightarrow \infty$. Ramanujan approximation is the most accurate solution for factorials, so g is the most accurate solution to the birthday problem. All together, this suggests that g is more accurate for smaller values for b and that accuracy for a approaches the accuracy of g as $b \rightarrow \infty$.

(3.): The accuracy tests for g, a measured with f_Γ :

For each iteration (i), first check if $k_{g(b_i, k)}$ and $a(b_i)$ result in the same $\text{floor}(k)$ since k is technically a discrete integer and if they're not equal, whichever function provides result closer to 50 % for the given expression is the most accurate function: $|f_\Gamma(b_i, k_r)|$ where k_r is the resultant k and is equal to either $k_{g(b_i, k)}$ or $a(b)$ and $b_i = i \times n$. The tests results show that g is more accurate around 16% (varying between 10-30%) of the time as $\Delta \rightarrow 0$, and $b \rightarrow \infty$. The small flaw with this is that since function g is the most accurate solution, even $f_\Gamma(b, k)$ function isn't going to show the exact accuracy. So the accuracy of g can only be determined accurately relative to a obviously less accurate function like function a .

The results:

```
g is more accurate: True
g is closer/equal to zero than a: 100
a is closer/equal to zero than g: 84
```

The accuracy of a is off mainly because the $\Delta \rightarrow \frac{\ln 2}{3}$ as $b \rightarrow \infty$. This is a strange phenomenon that is proven to be not related to function g since $f_\Gamma(b, k) \approx g(b, k) \approx a(b) + \frac{\ln 2}{3}$. a is always off by the same value because of how that a is derived. This means that a can be accurate 100% of the time as long as $a(b) + \frac{\ln 2}{3}$ is used rather than just $a(b)$ so that a returns k that's only off by a negligible amount for most contexts.

The code for this test is in `test/sec.py`.

$\therefore g(b, k)$ can be used to evaluate how many keys it would take so that 2 hashes have a 50% chance of being equal. $g(b, k)$ can be evaluated using the secant algorithm (`text/secant.py`)

After running `test/secant.py`, given the parameters:

Based on approximations in the Wikipedia Article: Birthday Problem, we can approximate x_0 and x_1

$$x_0 = \frac{1}{2} + \sqrt{\frac{1}{4} + 2 \times \ln(2) \times b} \text{ (due to Approximation of number of people)}$$

$$x_1 = \sqrt{b} \text{ (due to square approximation)}$$

error tolerance: $e = 1 \times 10^{-200}$

for $b = 4294967291^8$, we get $k_1 = 400651867432320527534628274526034254879$ for the root.

And for $b = 2^{256}$, we get $k_2 = 400651869298001176472314306405665023048$ for the root.

So then $\Delta k = k_2 - k_1 = 1865680648937686031879630768169 \approx 2^{101}$. Since the difference between k_1 and k_2 is negligible (2^{101} isn't big considering the magnitude of b). We can conclude that using a galois field doesn't increase risk of birthday attacks which justifies the use of galois fields security wise.

Assuming you can compute 1 trillion keys per second, it would still take 10^{27} seconds ($\frac{k_1}{1 \text{ trillion } \frac{\text{keys}}{\text{s}}}$) to try all keys so that there is a 50% chance of collision which is impossible.

2.2.2 Collision Attack

Most denial of service attacks related to encryption algorithms are based on brute-force methods. To see if this algorithm has a potential collision attack:

`transform(key1) = transform(key2)`

For example: $x + y = 16$

$$x, y \in \mathbb{Z}, 0 \leq x, y < 256$$

there are 17-combinations for x to satisfy this expression, and simultaneously, there are 17 combinations for y to satisfy the expression, so there is a total of 17 combinations.

But for $x + y = z$, there are 256 combinations to try. if the result of an arithmetic operation is known, there may be ways to get the same end-result with less combinations to brute-force. Knowing the value of z reduced the number of combinations by 15 times.

$\therefore$ if there is an operation that can provide the same output for a wide range of inputs, there can be a collision attack. Collision attacks can be used to derive the same transformed key using a different input key and decrypt the plaintext without actually having the original key.

In the context of this encryption algorithm (using addition as an example operation): keysize is 32-bytes

so for each byte n : $x_n + y_n = z_n$

$$x, y, z \in \mathbb{Z}, 0 \leq x, y, z < 256$$

Number of combinations can be represented by

$$\prod_{n=0}^{32-1} (z_n + 1)$$

so the number of combinations would be between a minimum of 32 combinations ($z_n = 0$ for all 32-bytes) up to a maximum of 2^{256} combinations ($z_n = 255$ for all 32-bytes) which can be brute forced for small z_n . So a simple addition is prone to collision attacks for $x_n + y_n = z_n$, where x, y are unknown. The use of galois field makes z_n even smaller. So even less combinations. Solution is to use operations that cannot be represented differently. e.g.

$$\sum_{i=0}^{z_n} x_n + y_n = z_n \implies \sum_{i=0}^{z_n} (z_n - i) + (i) = z_n$$

solves for all possible x, y values for each z_n . An addition operation can be represented differently to solve for 2 unknowns, while a good mix of ARX operations cannot be reverse engineered. This

is also the reason why pre-manipulating the key (using addition) before `transform()` isn't a good option. Since it provides a very obvious collision attack which makes it invalid even though pre-manipulation will provide a good avalanche effect for every single byte of the key (if 1-bit of any byte is changed, ciphertext changes completely).

So the final solution is to calculate sum of each 32-bit segment of the key (represented by k_i) in order to interlink them to make sure that every byte of the key offers the same avalanche effect:

$$\sum_{i=0}^7 k_i$$

To test if this offers enough collision resistance: think of this problem as an example:

1. $x + y = 16$ offers n=17 combinations
 2. $x + y + z = 16$ offers n=153 combinations (determined experimentally)
 3. $x + y + z + v = 16$ offers n=969 combinations (determined experimentally)
- $x, y, z, v \in \mathbb{Z}, 0 \leq x, y, z, v < 256$

So there has to be an equation or algorithm to summarize the relationship between number of variables (ℓ) and the sum of the addition operation (16);

Knowing that equation 1. is the simplest equation and it offers 17 combinations. Then if the rest of the equations are represented in 2-variable fashion. we can find number of combinations n :

For equation 2.: There are 3 ways to represent as 2-variable equation

$$x + y \quad x + z \quad y + z$$

For equation 3.: There are 6 ways to represent as 2-variable equation

$$x + y \quad x + z \quad x + v$$

$$y + z \quad y + v \quad z + v$$

The number of ways a multi-variable equation can be represented as a 2 variable equation can be summarized by the following:

$$\sum_{i=1}^{\ell-1} i$$

Using some number crunching and logic, we found that there is a direct correlation between the number of combinations and the ratio between the current number of ways to represent as 2-variable equation over the previous number of ways to represent as a 2-variable equation:

$$n_{\ell} \approx \frac{\sum_{i=1}^{\ell-1} i}{\sum_{i=1}^{\ell-2} i} \times k \quad \text{where } k \text{ is a multiplier: } k \in \mathbb{N}$$

Using more number crunching: We found the following recursive formula that finds the number of combinations that satisfies $x+y+\dots=16$:

$$n_\ell = (n_{\ell-1} \sum_{i=1}^x \frac{i-1}{x-1} + 17) \times 3 - 17 \times 3((x+1) \bmod 2)$$

where $x = \ell - 1$ and x should be incremented until correct answer is reached for $\ell \geq 6$ and $n_{\ell-1}$ is previous number of combinations. This formula doesn't translate to cases where the 2-variable equation doesn't have 17 combinations.

Simply put this equation couldn't be used accurately, it can only be an approximation. But upon further number crunching, we derived the following formula that satisfies all cases:

$$\prod_{i=0}^{\ell-1} \frac{(x+i)}{1+i}$$

where x is the number of combinations for 2 variable equations. e.g. for $a+b=16$, $x=16+1=17$.

This led me to develop this equation. The equation contains a costly operation (factorials) so it is computationally slower, but for approximation for really large numbers (e.g. sum of billions of variables), this could be better as it might be hard/unnecessary to calculate the exact answer using the formula when ℓ or c is unrealistically large:

$$\frac{(\ell+c)!}{c!\ell!}$$

Using the Ramanujan Approximation:

$$n! \approx \sqrt{\pi} \left(\frac{n}{e}\right)^n \sqrt[6]{8n^3 + 4n^2 + n + \frac{1}{30}}$$

We can derive an approximation equation for really large calculations:

$$\text{let } t = \frac{(\ell+c)!}{c!\ell!}$$

$$t \approx \frac{\sqrt[6]{\pi} \left(\frac{c+\ell}{e}\right)^{c+\ell} \sqrt[6]{8(c+\ell)^3 + 4(c+\ell)^2 + (c+\ell) + \frac{1}{30}}}{\sqrt[6]{\pi} \left(\frac{c}{e}\right)^c \sqrt[6]{8c^3 + 4c^2 + c + \frac{1}{30}} \sqrt[6]{\pi} \left(\frac{\ell}{e}\right)^\ell \sqrt[6]{8\ell^3 + 4\ell^2 + \ell + \frac{1}{30}}}$$

$$t \approx \pi^{-\frac{1}{2}} (c+\ell)^{c+\ell} e^{-c-\ell} c^{-c} e^c \ell^{-\ell} e^\ell \sqrt[6]{\frac{8(c+\ell)^3 + 4(c+\ell)^2 + (c+\ell) + \frac{1}{30}}{(8c^3 + 4c^2 + c + \frac{1}{30})(8\ell^3 + 4\ell^2 + \ell + \frac{1}{30})}}$$

$$t = \frac{(c+\ell)^{c+\ell}}{\sqrt{\pi} c^c \ell^\ell} \sqrt[6]{\frac{8(c+\ell)^3 + 4(c+\ell)^2 + (c+\ell) + \frac{1}{30}}{(8c^3 + 4c^2 + c + \frac{1}{30})(8\ell^3 + 4\ell^2 + \ell + \frac{1}{30})}}$$

The equation t defined above can be used to find the approximate number of combinations in an addition expression with ℓ number of variables with a sum of c .

Using the formula for the context of this encryption algorithm:
maximum 32-bit number:

$$c = 2^{32} - 1 = 0xffffffff$$

where $\ell = 8$ since 8 32-bit segments to the 256-bit key then, $x = c + 1$

the total number of combinations according to the equation is between 1 and
 $2871827628774669857283799072180574717903946432793745331030345747716374528 \approx 2.9 \times 10^{72}$

Which isn't possible to brute force provided key is random and not chosen to be a small value.

2.3 Joux's Attack: Multicollisions in Iterated Hash Functions

This attack requires knowing a collision: `transform( $k_1$ ) = transform( $k_2$ )`. Given a secure hashing algorithm, this isn't possible. Therefore, the Joux's attack [6] isn't possible. The only way to evaluate k_1 and k_2 would be using brute-force. That is mathematically proven in sections 2.2.1 and 2.2.2. This is also proven experimentally in `test/collision.cpp`. The values in `test/collision_report` show that over lots of iterations, getting the same hash is still impossible.

2.4 Related Key Attack

Related key attacks isn't possible considering that it requires that `transform( $k_1$ )` is similar to `transform( $k_2$ )` where k_1 and k_2 have a predictable difference (incremented by encryption index). But this is only under the assumption that the `transform` function is an irreversible hashing algorithm.

2.5 Time Based Attacks

Start from 32 zeros for a key, then incremented each byte by 1 until there are 32 255s and test again. This is tested in `test/benchmark` in function `benchmark_hash_time_attack_v()`. This test passed. The keys with small numbers have the same speed as keys with large numbers (This was never in doubt), therefore algorithm is proven to be constant time.

-----TESTING SIDE CHANNEL ATTACKS-----

```
TESTING TRANSFORMATION VERSION 1.0:  PASSED (NO SIDE CHANNEL): data is consistent
TESTING TRANSFORMATION VERSION 1.1:  PASSED (NO SIDE CHANNEL): data is consistent
TESTING TRANSFORMATION VERSION 2.0:  PASSED (NO SIDE CHANNEL): data is consistent
```

Tested for time-based side-channel attacks by comparing the computation time for different key values, If computation time for smaller key (32 0s) approximately equals to computation time for (32 255s) key, then data is consistent. This test is to see if there was a side-channel vulnerability (hardware/software) where someone approximates the value of the key by timing the calculation speed.

2.6 Linear Cryptoanalysis

Linear Cryptoanalysis is used to find linear connections between ciphertext and plaintexts.

Given that running `test/significant_perm_byte -r` provides that all bytes of the key provide approximately the same collision resistance and shows that there is no pattern. This means that modifying a certain byte isn't only affected some bytes but all of them without a pattern, therefore there are no significant bytes. And all bytes of the key affect the result equally. This is mainly due to the use of sum of k_i in the transformation algorithm.

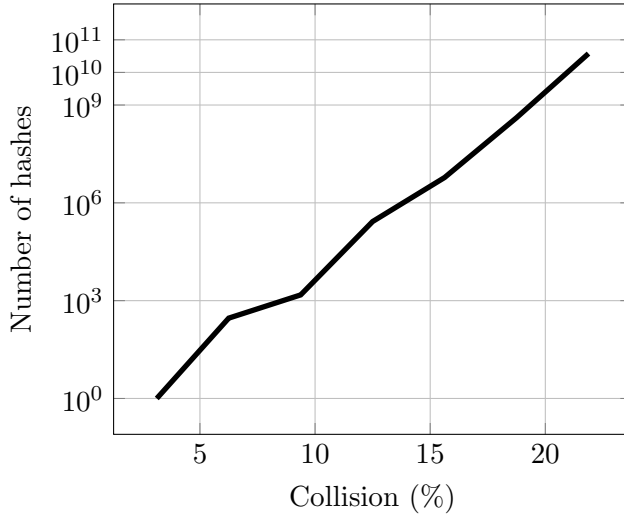
Here is data for SHA-256 hashing algorithm, what percentage of bytes can be equal, if this number gets to 100%, this means that there is a successful collision. i is the number of hashes that are generated to get $n\%$ equality.

%	Full-Time-Pad -(i)	SHA-256 (i)
3.125	1	2
6.25	291	6
9.375	1501	2648
12.5	267184	36595
15.625	5981439	6167198
18.75	408355721	530210645

Table 1: collision (%) VS. number of hashes

This data fluctuates a lot and therefore doesn't actually have any visible pattern, The numbers being similar to SHA-256 would be good news to suggest that the avalanche effect is real and that the math checks out for the birthday problem, it's not possible to find a collision.

Collision (%) VS. Number of Hashes for Full-Time-Pad



2.7 Differential Cryptanalysis

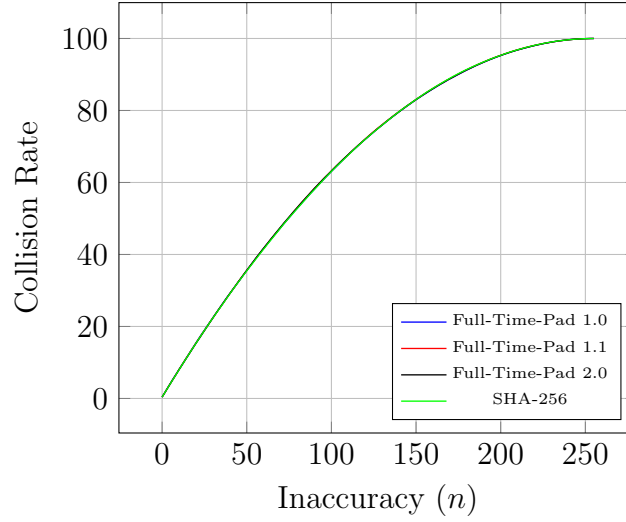
In `test/significant_perm_byte.cpp`, instead of comparing if values are equal, but rather approximately equal with accuracy of $\pm n$. This gave the same results as sha-256. And no specific bytes seemed to have followed a specific pattern. This means that there isn't a differential statistical pattern.

n	Full-Time-Pad (%)	SHA-256 (%)
0	0.3883	0.3923
5	4.2678	4.224
10	8.0342	8.0322
15	11.7421	11.7403
20	15.4626	15.4287
25	18.8071	18.8382

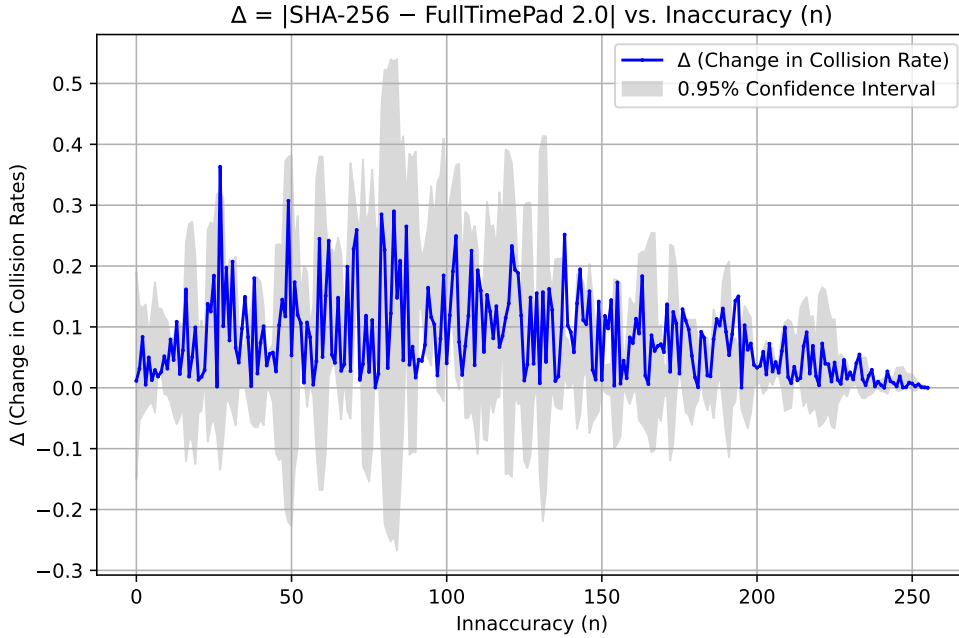
Table 2: Comparison Accuracy VS. Collision Rate

Results of collision rates are approximately equal given lower accuracy ($\pm n$). This suggests that there isn't any cryptoanalysis pattern. These numbers are average of 255 tries. Each try also yields similar results which means that there isn't a differential cryptoanalysis exploit regarding accuracy of comparison.

Figure 1: Collision Rate (%) vs. Inaccuracy (n)



These graphs demonstrates how lower accuracy of comparison results in higher collision rate (For both SHA-256 and Full-Time-Pad). The points for this graph is evaluated by running `test/significant_perm.byte -d -<version number>`. All 4 algorithms have the same result, thus they overlap.



This graph shows the difference (Δ) between SHA-256 hashing algorithm and the FullTimePad 2.0 which logically would be the most at risk version of Full-Time-Pad for security exploits. Since numbers on the y-axis are very small, this means that Δ is negligible and therefore there isn't a pattern.

The reason SHA-256 was chosen for this comparison is because this algorithm has proven itself to be secure.

2.8 Reverse Engineering the Transformation

this is demonstrated in the `test/reverse.cpp` code. The reversed key and key are very different.

2.9 Side-Channel Attacks

in FTP, the avalanche effect is strong and ARX already proved itself functional in cryptography. If key is potentially found due to a side-channel attack, then all plaintexts can be found. However this isn't a pressing issue as all cryptographic algorithms have this flaw. In contrast, without this flaw, a new key would have to be generated per plaintext like in One-Time-Pad which is inefficient.

2.10 Collision-Resistance

Collision resistance has been proven in the birthday problem section and the collision attack section. The collision resistance is on average about as low as the numbers in sha256. and don't have any particular patterns as proven in `test/significant_perm_byte.cpp`. This test file shows the collision rate as any byte of the 32-byte key is modified by 1-bit.

$$\text{collision_rate} = \frac{\text{\#collisions}}{32}$$

where `\#collisions` is the number of bytes where `previous keyi` and `modified keyi` are equal

2.10.1 Different Permutation Matrices

We tried permutation matrices that followed logic or randomness. But they didn't always offer the proper diffusion and collision resistance required to make a secure algorithm. The permutation matrix needs to be perfect so that the chances of collision (tested in `test/significant_perm_byte.cpp`) for every byte of the key should be around the same.

Using Bent functions to generate a non-uniform permutation matrix is also possible.

In terms of efficiency, the permutation matrix should be a uniform permutation matrix. The current matrix is non-uniform, which might provide better diffusion (more random), it also creates the issue of efficiency. So in terms of computation speed, reducing the number of times the memory is accessed is required. To achieve this, we will implement an updated version with uniform matrix and perform benchmark tests, based on results of both avalanche effect and cryptanalysis testing, we will keep/undo the tests.

2.10.2 Number of Rounds

Number of rounds for Version 1.0 & 1.1 is determined by the number of rows in the permutation matrix. As using a permutation twice will cancel out.

2.10.3 Constant - F_p - Prime Galois Field Size

every addition operation is done in a prime galois field as every $k_i \in \mathbb{F}_p$ where k_i is the i'th index of the 32-bit segmented key $p = 0xffffffffb$. p is the largest unsigned 32-bit prime number. It was evaluated using Fermat's Little Theorem:

Algorithm 1 Find the largest 32-bit prime number pseudo-code

```
1: Output: Biggest unsigned 32-bit prime number ( $p$ )
2: Begin
3:  $p = 0$ 
4: for  $i = 0xffffffff$  to 0 do
5:   if  $2^{i-1} \bmod i = 1$  then
6:      $p = i$ 
7:     break
8:   end if
9: end for
10: Return  $p$ 
```

2.10.4 Constant - r - Dynamic Rotation Constant

Cicadas only surface once every 17 years. Assuming that their predator only surfaces once every 2 years, they will only meet each other once every 34 years. Since 17 is a prime number, they will barely ever meet. However, if they surfaced once every 18 years, while their predator once every 2 years. Then every 18 years they would meet. This would also make it easier for the predator to synchronize their emergence timing.

let x be the time interval for when the cicadas surface

let y be the time interval for when the predators surface

We can conclude:

$GCD(x, y) = 1$, then they meet up every $x \times y$ years.

Whereas if $GCD(x, y) \neq 1$, then they meet up every x years (assuming $x > y$).

Bitwise rotation operation should also be done with either a prime number or a number where $GCD(x, y) = 1$ to maximize the collision resistance.

If not, then for example:

let w be 32-bit number

let $q = 8$ be rotation count

rotation can be represented by: $w = w \lll q$

If this rotation was repeated for 4 iterations, then the rotations would cancel out and rotated w (w') and w would be equal. Make sure that $GCD(q, 32) = 1$.

If there's only one rotation number, the likelihood of finding a pattern increases. Imagine a card dealer shuffling cards with two stacks, ensuring that one card from each stack is placed on top of the other in a consistent pattern. In this case, you could easily "unshuffle" the cards by simply separating them into two stacks, putting one card on one side and one card on the other. Using a single rotation like this is too simple, but if the cards were shuffled using a more complex method, the process becomes harder to reverse. Even though each method is reversible on its own, combining them makes it significantly more difficult to undo. The use of bitwise rotations on 32-bit segments combined with byte-level permutations creates excellent diffusion. The only potential

improvement would be to ensure that all bytes are interlinked solely through the bitwise rotations and permutations, rather than relying on the sum, as seen in the current transformation algorithm.

Rotation numbers that are selected are represented by r :

$$r = \begin{pmatrix} 23 & 5 & 17 & 31 & 13 \end{pmatrix}$$

There are 5 numbers for r , this is because $16 \bmod 5 \neq 0$, after 16 iterations in the transformation algorithm, all values of k aren't rotated the same number of times.

If you were to rotate 32-bit unsigned integer (x)

$$\sum_{i=0}^4 r_i = 89$$

number of times, the net rotation would be 89 (a prime number). $x \ggg 89$ would produce a net rotation of $x \ggg 25$, since $89 \bmod 32 = 25$. Knowing that $GCD(25, 32) = 1$, the minimum number of times where the expression $x \ggg 89$ is repeated till rotated x equals x is $89 \times 32 = 2848$. This means that only after 2848 iterations would rotated x equal x . Whereas if there was a singular number for r , the number of rotations for rotated x to equal x would be $r \times 32$ (e.g. $r = 23, 23 \times 32 = 736$).

$\therefore$ there needs to be multiple numbers for r .

It is designed so that $|r_i - r_{i-1}|$ results in a different magnitude each time so that the net rotation is a bigger number (more rotation means better diffusion/confusion).

For the derivation, the numbers started with prime numbers, all prime numbers from 0-32, but most were eliminated due to similar values for $|r_i - r_{i-1}|$ results (e.g. $23 - 5 = 18, |5 - 17| = 12$ aren't too close). A good net rotation would be prime and without similarities to net rotations. In the transformation algorithm, There are bitwise right rotate ($\ggg$) and bitwise left rotate ($\lll$), this is done to properly mix the results.

3 Hashing

The 'hashing' algorithm is used for manipulating the key each time with proper confusion and avalanche effect. The main reason why the key can be reused is due to the algorithm's strong avalanche effect as well as the lack of statistical patterns with collision rates as proven in the unit tests, `test/significant_perm_byte.cpp` which shows if some bytes have more or less collision resistance.

To maximize the effects of confusion using the current logic, 4 out of 8 values k (32-bit segmented key) should be modified per iteration (half of key). So each byte of the key is transformed a total of 8 times ($\frac{16 \text{ iterations}}{8 \div 4}$). since $16 \bmod 4 = 0$, all bytes are transformed the same number of times which would also tarnish the possibility of a statistical exploit that could target less transformed bytes of the key.

Half of the key should be modified per iteration because classical cryptographic algorithms (non-quantum) rely on computational complexity for their security. So more transformation the better. But the most minimum work to get a secure transformed key is enough, therefore it's the goal. this was determined experimentally; testing collision resistance of each byte upon modifying the transformation algorithm.

We can represent the transformation algorithm with a game of cards. Imagine that 8 cards are randomly picked in a pile of 2^{256} cards, each card is cut up into 4 equal pieces, the cut up pieces are shuffled using a permutation algorithm while also replaced with other cut up values from the pile of cards and simultaneously, some pieces are cut up to 2 pieces with magnitude of a prime number (e.g. 5mm, other pieces being 32-5mm if card is 32mm). If this process repeated for 16 iterations, could you recover the original cards? Probably not.

3.1 Dynamic Matrix Permutation

Considering the use of `reinterpret_cast` for representing the bytearray (key) as 32-bit segments (for confusion operations), the endiannes of the computer will affect the matrix. the matrix given in this documentation is the big endian version as the algorithm is designed to be big-endian. The little endian version requires a new permutation matrix where the byte order every 32-bits is reversed. This is in the C++ implementation of `Full-Time-Pad` and the python Deravation program mentioned below.

This generates a non-uniform permutation matrix. Which is a "key" concept in uniform randomization of the transposition of bytes.

3.1.1 Deravation

Algorithm 2 Dynamic Permutation Matrix Deravation Pseudo-code

```
1: Input: an array of incrementing numbers (0-31)  $A$ 
2: Output: Most Efficient Permutation Matrix  $V$  ( $16 \times 32$ )
3: Begin
4:  $P \leftarrow \text{copy of } A$ 
5: for  $k = 0$  to 4 do
6:   for  $i = 0$  to 8 do
7:      $P_i \leftarrow A_{i \times 4}$ 
8:      $P_{i+8} \leftarrow A_{i \times 4 + 1}$ 
9:      $P_{i+16} \leftarrow A_{i \times 4 + 2}$ 
10:     $P_{i+24} \leftarrow A_{i \times 4 + 3}$ 
11:   end for
12:    $A \leftarrow \text{copy of } P$ 
13:    $V.append(P)$ 
14:    $C \leftarrow \text{copy of } P$ 
15:   for  $m = 0$  to 3 do
16:     for  $i = 0$  to 8 do
17:       for  $n = 0$  to 4 do
18:          $P_{i \times 4 + n} \leftarrow C_{(1+n+m) \bmod 4 + i \times 4}$ 
19:       end for
20:     end for
21:      $V.append(P)$ 
22:   end for
23:    $A \leftarrow \text{copy of } P$ 
24: end for
25: Return  $V$ 
```

Python code is in the test/perm.py

3.1.2 Dynamic Permutation Matrix Values

$$\left(\begin{array}{cc} 0 & 4 & 8 & 12 & 16 & 20 & 24 & 28 & 1 & 5 & 9 & 13 & 17 & 21 & 25 & 29 & 2 & 6 & 10 & 14 & 18 & 22 & 26 & 30 & 3 & 7 & 11 & 15 & 19 & 23 & 27 & 31 \\ 4 & 8 & 12 & 0 & 20 & 24 & 28 & 16 & 5 & 9 & 13 & 1 & 21 & 25 & 29 & 17 & 6 & 10 & 14 & 2 & 22 & 26 & 30 & 18 & 7 & 11 & 15 & 3 & 23 & 27 & 31 & 19 \\ 8 & 12 & 0 & 4 & 24 & 28 & 16 & 20 & 9 & 13 & 1 & 5 & 25 & 29 & 17 & 21 & 10 & 14 & 2 & 6 & 26 & 30 & 18 & 22 & 11 & 15 & 3 & 7 & 27 & 31 & 19 & 23 \\ 12 & 0 & 4 & 8 & 28 & 16 & 20 & 24 & 13 & 1 & 5 & 9 & 29 & 17 & 21 & 25 & 14 & 2 & 6 & 10 & 30 & 18 & 22 & 26 & 15 & 3 & 7 & 11 & 31 & 19 & 23 & 27 \\ 12 & 28 & 13 & 29 & 14 & 30 & 15 & 31 & 0 & 16 & 1 & 17 & 2 & 18 & 3 & 19 & 4 & 20 & 5 & 21 & 6 & 22 & 7 & 23 & 8 & 24 & 9 & 25 & 10 & 26 & 11 & 27 \\ 28 & 13 & 29 & 12 & 30 & 15 & 31 & 14 & 16 & 1 & 17 & 0 & 18 & 3 & 19 & 2 & 20 & 5 & 21 & 4 & 22 & 7 & 23 & 6 & 24 & 9 & 25 & 8 & 26 & 11 & 27 & 10 \\ 13 & 29 & 12 & 28 & 15 & 31 & 14 & 30 & 1 & 17 & 0 & 16 & 3 & 19 & 2 & 18 & 5 & 21 & 4 & 20 & 7 & 23 & 6 & 22 & 9 & 25 & 8 & 24 & 11 & 27 & 10 & 26 \\ 29 & 12 & 28 & 13 & 31 & 14 & 30 & 15 & 17 & 0 & 16 & 1 & 19 & 2 & 18 & 3 & 21 & 4 & 20 & 5 & 23 & 6 & 22 & 7 & 25 & 8 & 24 & 9 & 27 & 10 & 26 & 11 \\ 29 & 31 & 17 & 19 & 21 & 23 & 25 & 27 & 12 & 14 & 0 & 2 & 4 & 6 & 8 & 10 & 28 & 30 & 16 & 18 & 20 & 22 & 24 & 26 & 13 & 15 & 1 & 3 & 5 & 7 & 9 & 11 \\ 31 & 17 & 19 & 29 & 23 & 25 & 27 & 21 & 14 & 0 & 2 & 12 & 6 & 8 & 10 & 4 & 30 & 16 & 18 & 28 & 22 & 24 & 26 & 20 & 15 & 1 & 3 & 13 & 7 & 9 & 11 & 5 \\ 17 & 19 & 29 & 31 & 25 & 27 & 21 & 23 & 0 & 2 & 12 & 14 & 8 & 10 & 4 & 6 & 16 & 18 & 28 & 30 & 24 & 26 & 20 & 22 & 1 & 3 & 13 & 15 & 9 & 11 & 5 & 7 \\ 19 & 29 & 31 & 17 & 27 & 21 & 23 & 25 & 2 & 12 & 14 & 0 & 10 & 4 & 6 & 8 & 18 & 28 & 30 & 16 & 26 & 20 & 22 & 24 & 3 & 13 & 15 & 1 & 11 & 5 & 7 & 9 \\ 19 & 27 & 2 & 10 & 18 & 26 & 3 & 11 & 29 & 21 & 12 & 4 & 28 & 20 & 13 & 5 & 31 & 23 & 14 & 6 & 30 & 22 & 15 & 7 & 17 & 25 & 0 & 8 & 16 & 24 & 1 & 9 \\ 27 & 2 & 10 & 19 & 26 & 3 & 11 & 18 & 21 & 12 & 4 & 29 & 20 & 13 & 5 & 28 & 23 & 14 & 6 & 31 & 22 & 15 & 7 & 30 & 25 & 0 & 8 & 17 & 24 & 1 & 9 & 16 \\ 2 & 10 & 19 & 27 & 3 & 11 & 18 & 26 & 12 & 4 & 29 & 21 & 13 & 5 & 28 & 20 & 14 & 6 & 31 & 23 & 15 & 7 & 30 & 22 & 0 & 8 & 17 & 25 & 1 & 9 & 16 & 24 \\ 10 & 19 & 27 & 2 & 11 & 18 & 26 & 3 & 4 & 29 & 21 & 12 & 5 & 28 & 20 & 13 & 6 & 31 & 23 & 14 & 7 & 30 & 22 & 15 & 8 & 17 & 25 & 0 & 9 & 16 & 24 & 1 \end{array} \right)$$

3.2 Confusion - ARX

For the confusion aspect of the key transformation algorithm, ARX operations are used.

3.2.1 A - Modular Addition

For the modular addition, a constant array of 8 values is used (A):

```

encryption index >> 32
encryption index & 0xffffffff
0x119f904f
0x73d44db5
0x3918fa83
0x5546b403
0x216c46df
0x64997dfd

```

A is derived as follows:

Start with the message: "**cipher**" (6 digit message with 6 uneiqe characters)

Take every ascii encoding of each character of the message and calculate to the 32nd power and take the first 32-bits:

Example:

"c": 99 in ascii encoding

In python: `hex(9932)[10:]` = 0x119f904f

The word **cipher** and $\mathbf{A}_i$ is arbitrary. In other words, it's "nothing up my sleeve numbers". This is an initial array. In the transformation algorithm, every $\mathbf{A}_i$ is modified twice (in 16 iterations). This provides the confusion. The reason why the first 2 values are the 64-bit encryption index is because this was the most efficient way to diffuse in the encryption index.

3.2.2 R - Bitwise Rotation

"Using the values determined for r (as discussed in Section 2.10.4), rotate one value to the right, another to the left, and then the third to the right again. The reasoning behind alternating the rotation directions goes back to the card analogy. Imagine two stacks of cards. One on the left and the other on the right. To shuffle them efficiently, a logical approach would be to alternately take a card from the left stack and place a card from the right stack on top. For a "randomized" shuffle, the best strategy would be to take n th card from alternating sides, where n is a prime number, to minimize collisions."

3.2.3 X - XOR

Some of the values are xored because xor operator offers 256 combinations per byte unlike unsigned addition which mostly offers less (section 2.2.2). This will bring more possibilities which in return limits brute-forcing attacks. It is avoided when there is a good possibility of xoring the same 2 values ($x \oplus x$), this is why the sum of k_i is used rather than xor.

4 Performance

In terms of performance, the transformation version 1.0 is quite slow compared to ChaCha20 and AES. The main reason for the poor performance is the permutation. Version 2.0 of the transformation algorithm could use further optimizations in later versions of the Full-Time-Pad cipher which is the fastest yet secure version of the algorithm. Not all versions value speed, some are purely focused on security. Sometimes when dealing with things like passwords, we would recommend using slower algorithms to reduce brute-forcing possibilities.

Benchmark tests show the following results:

-----TESTING SPEED-----

TESTING TRANSFORMATION VERSION 1.0:

Computation Time for 1000000 key transformations: 0.641983 seconds

Hashrate: 1557674 keys per second

TESTING TRANSFORMATION VERSION 1.1:

Computation Time for 1000000 key transformations: 0.526699 seconds

Hashrate: 1898618 keys per second

TESTING TRANSFORMATION VERSION 2.0:

Computation Time for 1000000 key transformations: 0.059143 seconds

Hashrate: 19178011 keys per second

On the same device (12th Gen Intel(R) Core(TM) i7-12700H, DDR5 RAM). The ChaCha20 benchmark tests yield the following results:

Time taken to encrypt 1000000 32-byte messages: 0.160442 seconds

Encryption rate: 6232781 keys per second

This means that Full-Time-Pad transformation version 2.0 is more than 3 times faster than ChaCha20. However, ChaCha20 has 64-byte keystreams while my algorithm has 32-byte. So Mine is 3x faster for encrypting unique plaintexts under 32 bytes (For long plaintexts, 1.5x). These tests are under the assumption that there are full optimizations possible (modern hardware).

One must also consider that my algorithm could be redesigned to generate 64-byte blocks faster than ChaCha20 as longer keystream doesn't mean a linear increase in time. This is a potential upgrade in future version of this algorithm.

Also, for Versions 1.0 & 1.1, The poor performance is mainly due to the poor implementation. Implementation could be improved, yet we don't think speed would be affected too significantly.

5 Cipher

5.1 Transformation - Version 1.0 - Prioritize Complexity

Algorithm 3 The key transformation operation per encryption

```

1: Input: 32-byte bytearray key (key), 64-bit encryption_index
2: Output: Transformed 32-byte bytearray key ( $k$ )
3: Begin
4: // keysize = 32, length of input key is 32-bytes
5: uint32_t *k  $\leftarrow$  endian_8_to_32_arr(key)
6: for  $i = 0$  to 16 do
7:   index  $\leftarrow i \ll 2$ 
8:   i1mod  $\leftarrow$  index mod 8
9:   i2mod  $\leftarrow$  (index + 1) mod 8
10:  i3mod  $\leftarrow$  (index + 2) mod 8
11:  i4mod  $\leftarrow$  (index + 3) mod 8
12:  imod8  $\leftarrow i$  mod 8
13:  imod9  $\leftarrow (i + 1)$  mod 8
14:  rmod  $\leftarrow i$  mod 5
15:
16:  // No unwanted overflow, so convert to uint64_t
17:   $k_{i1mod} \leftarrow (\text{uint64\_t})k_{i1mod} + A_{imod8} + k_{i1mod} \ggg r_{rmod} \pmod{p}$ 
18:  sum  $\leftarrow \sum_{j=0}^8 k_j \pmod{p}$ 
19:   $A_{imod9} \leftarrow A_{imod9} \oplus \text{sum}$ 
20:   $k_{i2mod} \leftarrow (\text{uint64\_t})k_{i2mod} + A_{imod9} + k_{i2mod} \lll r_{rmod} \pmod{p}$ 
21:   $A_{imod8} \leftarrow A_{imod8} \oplus (\text{uint64\_t})k_{i2mod} + k_{i1mod} \ggg r_{(i+1) \bmod 5} \pmod{p}$ 
22:   $k_{i3mod} \leftarrow (\text{uint64\_t})A_{imod8} \oplus k_{i3mod} + A_{imod9} \oplus k_{i4mod} \pmod{p}$ 
23:   $k_{i4mod} \leftarrow (\text{uint64\_t})A_{imod8} \oplus k_{i4mod} + A_{imod9} \oplus k_{i3mod} \pmod{p}$ 
24:
25:  // Permutate the bytearray key
26:  key  $\leftarrow$  dynamic_permutation(key,  $i$ );
27:  increment encryption_index by 1 for each plaintext
28: end for
29: Return key

```

5.2 Transformation - Version 1.1 - Prioritize Speed & Complexity

This version of the transformation offers more speed less complexity. This is preferred to the previous option since the other has more unnecessary operations that add complexity but isn't really needed, the previous option is better suited for when speed doesn't matter as much as security, or when slower algorithms are appreciated.

Algorithm 4 The key transformation operation per encryption

```
1: Input: 32-byte bytearray key (key), 64-bit encryption_index
2: Output: Transformed 32-byte bytearray key ( $k$ )
3: Begin
4: // keysize = 32, length of input key is 32-bytes
5: uint32_t *  $k \leftarrow \text{endian\_8\_to\_32\_arr}(\text{key})$ 
6: for  $i = 0$  to 16 do
7:    $\text{index} \leftarrow i \ll 2$ 
8:    $i1\text{mod} \leftarrow \text{index} \bmod 8$ 
9:    $i2\text{mod} \leftarrow (\text{index} + 1) \bmod 8$ 
10:   $i3\text{mod} \leftarrow (\text{index} + 2) \bmod 8$ 
11:   $i4\text{mod} \leftarrow (\text{index} + 3) \bmod 8$ 
12:   $i\text{mod}8 \leftarrow i \bmod 8$ 
13:   $i\text{mod}9 \leftarrow (i + 1) \bmod 8$ 
14:   $r\text{mod} \leftarrow i \bmod 5$ 
15:
16:  // No unwanted overflow, so convert to uint64_t
17:   $k_{i1\text{mod}} \leftarrow (\text{uint64\_t})k_{i1\text{mod}} + A_{i\text{mod}8} + k_{i1\text{mod}} \ggg r_{r\text{mod}} \pmod{p}$ 
18:   $\text{sum} \leftarrow \sum_{j=0}^8 k_j \pmod{p}$ 
19:   $A_{i\text{mod}9} \leftarrow A_{i\text{mod}9} \oplus \text{sum}$ 
20:   $k_{i2\text{mod}} \leftarrow (\text{uint64\_t})k_{i2\text{mod}} + A_{i\text{mod}9} + k_{i2\text{mod}} \lll r_{r\text{mod}} \pmod{p}$ 
21:   $A_{i\text{mod}8} \leftarrow A_{i\text{mod}8} \oplus k_{i2\text{mod}} \pmod{p}$ 
22:   $k_{i3\text{mod}} \leftarrow A_{i\text{mod}8} \oplus k_{i3\text{mod}} \pmod{p}$ 
23:   $k_{i4\text{mod}} \leftarrow A_{i\text{mod}8} \oplus k_{i4\text{mod}} \pmod{p}$ 
24:
25:  // Permutate the bytearray key
26:  key  $\leftarrow \text{dynamic\_permutation}(\text{key}, i);$ 
27: end for
28: increment encryption_index by 1 for each plaintext
29: Return key
```

5.3 Transformation - Version 2.0 - Prioritize Speed

This version of the transformation prioritizes speed. This is preferred to the previous option if complexity doesn't matter as much. This is still secure, but not as secure as Version 1.0. This version is the updated Version 1.1, the main difference is removing the galois field as this still provides a good confusion and making the algorithm have only 10 rotations (Determined experimentally).

Algorithm 5 The key transformation operation per encryption

```
1: Input: 32-byte bytearray key (key), 64-bit encryption_index
2: Output: Transformed 32-byte bytearray key ( $k$ )
3: Begin
4: // keysize = 32, length of input key is 32-bytes
5: uint32_t *k  $\leftarrow$  endian_8_to_32_arr(key)
6: for  $i = 0$  to 10 do
7:   index  $\leftarrow i \ll 2$ 
8:   i1mod  $\leftarrow$  index mod 8
9:   i2mod  $\leftarrow$  index + 1) mod 8
10:  i3mod  $\leftarrow$  index + 2) mod 8
11:  i4mod  $\leftarrow$  index + 3) mod 8
12:  imod8  $\leftarrow i$  mod 8
13:  imod9  $\leftarrow (i + 1)$  mod 8
14:  rmod  $\leftarrow i$  mod 5
15:
16:   $k_{i1mod} \leftarrow k_{i1mod} + A_{imod8} + k_{i1mod} \ggg r_{rmod}$ 
17:  sum  $\leftarrow \sum_{j=0}^8 k_j$ 
18:   $A_{imod9} \leftarrow A_{imod9} \oplus$  sum
19:   $k_{i2mod} \leftarrow k_{i2mod} + A_{imod9} + k_{i2mod} \lll r_{rmod}$ 
20:   $A_{imod8} \leftarrow A_{imod8} \oplus k_{i2mod}$ 
21:   $k_{i3mod} \leftarrow A_{imod8} \oplus k_{i3mod}$ 
22:   $k_{i4mod} \leftarrow A_{imod8} \oplus k_{i4mod}$ 
23:
24:  // Permutate the bytearray key only 2 times, don't permute before last iteration
25:  if  $i$  mod 4 = 0 and  $i \neq 8$  then
26:    key  $\leftarrow$  dynamic_permutation(key,  $i$ );
27:  end if
28: end for
29: increment encryption_index by 1 for each plaintext
30: Return key
```

5.4 dynamic_permutation

Algorithm 6 The `dynamic_permutation()` in the transformation function

```
1: Input: 32-byte bytearray ( $B$ ), permutation matrix ( $V$ ), iteration index ( $i$ )
2: Output: 32-byte permuted bytearray ( $B$ )
3: let  $P$  be placeholder vector declared with length of 32-bytes
4: for  $j = 0$  to 32 do
5:      $P_j = B_{V_{i_j}}$ 
6: end for
7: Return  $P$ 
```

This endiannes conversion is designed to convert a little-endian reinterpret casted pointer into the big-endian version so that both little-endian and big-endian systems are compatible.

5.5 endian_8_to_32_arr

Algorithm 7 The `endian_8_to_32_arr()` in the transformation function

```
1: Input: 32-byte bytearray ( $B$ )
2: Output: 32-byte bytearray ( $B$ ) permuted to flip endiannes if necesarry
3: if big endian system then
4:     for  $j = 0$  to 32 do
5:         swap  $B_i$  with  $B_{i+3}$ 
6:         swap  $B_{i+1}$  with  $B_{i+2}$ 
7:     end for
8: end if
9: Return reinterpret_cast <uint32_t*>( $B$ )
```

5.6 Key Reuse

In order to reuse the key, there is an encryption index which is a 64-bit nonce that is incremented per encryption in order to keep the transformed key unique. The encryption index is infused in the encryption algorithm in an efficient manner, the first 2 values is the encryption index. It's the first 2, not the last 2 because indexing in the transformation algorithm uses the first 2 indexes of $\mathbf{A}$ so this would provide more iterations to diffuse in the encryption index. The encryption index could also be randomly generated each time. If it's incremented it's necessary to make sure that the same encryption index is incremented when the plaintext is long (not just incremented per message but per 32-byte segment of every message sent.)

5.6.1 Multi-Threading

When multiple threads are encrypting using the same key and the encryption index is incremented per encryption, the encryption index needs to be an atomic variable. A substitute is to use a random encryption index, but this could lead to collisions since root k for $g(2^{64}, k) = 5,056,937,540$ where g is a solution to the birthday problem defined in subsubsection 2.2.1. This means that there is

a 50% chance of collision when 5 billion 64-bit nonces are used, since a new nonce is needed per 32-byte message, 161,822,001,280 bytes of data would need to be encrypted for a succesful brute-force to be 50% likely. In practice, using random nonces increase chances of collisions by a lot. So the safest practice would be to increment the encryption index and keep it as an atomic variable for multi-threading. If using an atomic number isn't possible (e.g. threads cannot communicate efficiently), then use one the following methods, the first one is simpler and faster:

Algorithm 8 Unique nonces for Multi-threading without thread communication

```

1: Input: thread count ( $t$ ), thread ID ( $n$ )
2: if first encryption_index then
3:   encryption_index =  $n$ 
4: else
5:   Add  $t$  to encryption_index
6: end if
7: Output: Nonce (encryption_index)

```

Algorithm 9 Unique nonces for Multi-threading without thread communication

```

1: Input: thread count ( $t$ ), thread ID ( $n$ ), last encryption index used on this thread ( $m$ ),  

    $m > 0$ , ordered set of  $t$  prime numbers ( $P$ )
2: let encryption_index  $\leftarrow n + m + 1$ 
3: let  $p \in P, p \neq P_n$ 
4: while gcd(encryption_index,  $p$ )  $> 1$  do
5:   encryption_index  $+= P_n$ 
6: end while
7: Output: Nonce (encryption_index)

```

These algorithm are only valid if number of threads used is not growing. Otherwise, every time a new thread is created, the algorithm needs to be recalled with a new t . If thread count might need to grow, just set thread count to maximum possible number when you initially use this algorithm.

Algorithm-9 proves itself redundant since `encryption_index` always ends up being a power of P_n (e.g. for $P_n = 3$, `encryption_index` = 3, 9, 27, 81...). But exponential growth doesn't work for long plaintexts. $\therefore$ Use **Algorithm 8** for generating incremental nonces without thread communication.

5.7 Long Plaintexts

When there are long plaintexts used, the encryption index should be incremented by 1 the same way as key reuse. It might be logical to randomly generate the encryption index per plaintext so that you don't have to keep track of incrementing the encryption index for all plaintexts but rather per plaintext. This isn't an issue in the algorithm itself but rather a side-channel exploit if you don't update encryption index. My implementation of the algorithm doesn't impose this problem as long as you don't use the same key for 2 `FullTimePad` Objects.

5.8 Encryption/Decryption operation

Encryption operation is the same as decryption

Algorithm 10 Encrypt/Decrypt transform Function

```
1: Input: plaintext ( $p$ ), length ( $\ell$ ), 32-byte original key ( $k$ ), encryption index/nonce  
   (encryption_index)  
2: Output: ciphertext ( $c$ )  
3: for each 32-byte segment of plaintext do  
4:   // create a copy of unmodified  $k$ , and transform it using an incrementing unique  
   encryption_index  
5:   // Then, encrypt using plaintext xor transformed key = ciphertext  
6:   copy = copy(k)  
7:    $c \leftarrow c || p \oplus \text{transformation}(\text{copy}, \text{encryption\_index})$   
8: end for  
9:  
10:  
11: // For the remainder:  
12:  $c \leftarrow c || p \oplus \text{transformation}(\text{copy}, \text{encryption\_index})$   
13: Return  $c$ 
```

References

- [1] Claude Shannon. *A Mathematical Theory of Cryptography*. 1945.
- [2] Michael Sung. Feb. 2000. URL: <http://www.ai.mit.edu/projects/aries/papers/writeups/darkman-writeup.pdf>.
- [3] Dec. 2001. URL: <http://palms.ee.princeton.edu/PALMSopen/lee01efficient.pdf>.
- [4] Mar. 2025. URL: https://en.wikipedia.org/wiki/Single_instruction,_multiple_data.
- [5] May 2025. URL: https://en.wikipedia.org/w/index.php?title=Birthday_problem&oldid=1291691534.
- [6] Antoine Joux. *Multicollisions in iterated hash functions. Application to cascaded constructions*. Mar. 2025. URL: <https://www.iacr.org/archive/crypto2004/31520306/multicollisions.pdf>.